

System Requirements: Intelligent Teleoperation Interface Design

Group 9

Mazen Mohamed Nasr	2401811
Mark Rafik Hanna	2401812
Yahia Mohamed Mady	2401813
Aras Sulaiman	2406393
Muhammad Ahmed Khan	2404472
Mohith Surendra	2401477
Amer Haitham Harfoush	2401810

6th February 2025

Table of Contents

Table of Contents	2
1 Abstraction.....	3
2 Introduction	3
2.1 Purpose	4
2.2 Scope	4
2.3 Overview	5
3 Requirements Analysis and Specification.....	5
3.1 Requirements Elicitation Methods.....	5
3.2 System Architecture.....	5
3.4 Functional Requirements (Product Back- log).....	6
3.5 Non-Functional Requirements	7
4 Testing	7
4.1 Testing Schedule	7
4.1.1 Unit Testing.....	7
4.1.2 Integration Testing	8
4.1.3 System Testing	8
4.1.4 Acceptance Testing.....	8
4.2 Test Cases	8
4.2.5 UI Testing	8
4.2.6 AI Model Testing	8
4.2.7 ROS Integration Testing.....	8
4.3 Testing Schedule Timeline.....	9
4.4 Tools and Resources	10
5 Project Planning	10
5.1 Project Management Structure/Methodology.....	11
5.1.1 Agile Methodology	11
5.1.2 Waterfall Methodology	11
5.1.3 Hybrid Methodology	11
5.1.4 Lean Methodology.....	12
5.2 Project Planning Tools	12
5.2.1 Jira.....	12
5.2.2 GitLab	12
6 References	14

1 Abstraction

Teleoperation is essential for mitigating human constraints in dangerous settings, facilitating the remote handling of robotic systems for various applications in industry, research, and exploration. This initiative centers on creating an advanced teleoperation interface for the Dobot robotic arm, incorporating cutting-edge AI technologies such as machine vision, image classification, large language models (LLMs), and the Robotics Operating System (ROS). The objective of the system is to enhance human-robot interaction through voice commands, gesture recognition, and immediate color detection and blending, thereby increasing the efficiency and adaptability of teleoperated robotic systems. By utilizing AI and computer vision, the project seeks to minimize the dependence on costly hardware components, substituting traditional sensors with economical cameras and processing devices.

The architectural design of the system takes a methodical approach, segmenting the interface into input, processing, and output layers to guarantee smooth interaction. Key functional requirements encompass gesture-based control, voice-activated shape creation, and automated color mixing, emphasizing performance, user-friendliness, and reliability. The project adopts Agile methodologies for an iterative development process, enabling responsiveness to real-world scenarios and user preferences. Testing stages comprise unit, integration, and system validation, utilizing tools such as ROS Gazebo and PyTest to enhance performance prior to deployment. By merging AI-driven automation with a user-friendly interface, this project aspires to establish a new benchmark for effective and intelligent teleoperation in robotics.

2 Introduction

Teleoperation is a necessity to enable us to break the limits that restrict us like safety and physical limitations. A robot is able to be in hazardous environments with extreme conditions that humans can't stand being in. By Using Teleoperation Robotics technology we unleash the capabilities of our research and work, like having robots on Mars (spirit and opportunity) the two

exploration rovers that have been on mars for over 21 years. Not only having the robots in a harsh environment but also having them continuously working without the need to rest much (unless overheated) because unlike humans they can be electrically powered in a factory increasing production rate and also can be using solar energy.

2.1 Purpose

The intelligent teleoperation interface design aims to build a system that allows efficient remote manipulation of a Dobot robotic arm. By integrating the human- friendly interface with some advanced and intelligent systems. The project looks forward to minimizing the gap between human operators and robotic systems. Allowing the Robot to work intelligently, responding to voice commands and interacting with humans using gesture based communication.



2.2 Scope

The scope of this project is to develop an understandable teleoperation interface that integrates Machine Vision, Image Classification, Large Language Models, ROS (Robotics Operating System) to boost the human-robot interaction. We will focus mainly on three tasks:

- **Hand Gestures Recognition:** Designing algorithms that detect hand gestures in real time and execute some actions based on it.
- **Shapes drawing:** Developing routines that understand user desires about drawing some shapes and draw them.
- **Color detection and mixing:** Creating a model that blends multiple colors together to produce another color based on the user command.

2.3 Overview

Although the conceptualization and development of teleoperation started as early as the 1900s, the first master–slave teleoperation system was developed during the mid-1940s.

After research it has been found that most of the commonly used methods in teleoperation rely mostly on sensor and embedded systems. However, it is from the interests of this project to replace that with computer vision and AI model, that will have a huge effect on the cost in the industrial field by massively decreasing the cost of the hardware components as most of the sensors and chips be replaced with a camera and a single processor.

In this project we will be using the Dobot which was designed to be a robotic arm for teaching that we are using to mimic any robot arm in most industries. The Dobot robotic arm is a versatile, high-precision robotic manipulator designed for education, research, and industrial applications. It supports multi-axis movement, ROS integration, and various end-effectors, making it ideal for automation, teleoperation, and AI-driven tasks. Which makes it the best option for our project.

3 Requirements Analysis and Specification

3.1 Requirements Elicitation Methods

- **Weekly meetings** with the project supervisor to define expectations.
- **Literature review** focusing on AI integration, ROS control, and computer vision.
- **Simulation and testing** in Gazebo to identify and solve unforeseen problems.

3.2 System Architecture

The system architecture is divided into three layers:

1. Input Layer

- Hand Gesture Recognition.
- Voice Command Recognition (using LLM).
- Color Input (using a camera).

2. Processing Layer:

- LLM Module for processing natural language commands.
- Gesture Control Module to translate hand gestures into robot actions.
- Color Prediction Module for describing color mixing outcomes.

3. Output Layer:

- Dobot Robot for object movement, drawing, and color mixing.
- GUI for real-time status updates.
- Audio feedback to confirm user commands.

The hand gesture project is going to take the input from a camera and splitting the hand into landmarks which is to identify each point on the hand (index tip, thumb bas, hand base and each knuckle) then processing it through a Gesture control Module to figure out the order from the human to output the action on the Dobot. The Voice Command recognition will use both mic and camera, A mic to take the speech then convert it to text which is going to be processed by the LLM and A Camera to be able to interact with the environment then the Dobot is going to carry out the output physically.

3.4 Functional Requirements (Product Back- log

ID	Description	Priority	Acceptance Criteria
1	Recognize hand gestures	High	Robot moves based on hand gestures
2	Process voice commands via LLM	High	Interprets shape names
3	Generate and display mixed color result	Medium	Shows mixed color
4	Object pickup and drop	High	Robot performs movement correctly
5	Draw shapes by voice command	High	Robot draws the requested shape
6	Provide voice feedback for actions	Low	System confirms actions via audio

3.5 Non-Functional Requirements

ID	Description	Priority	Acceptance Criteria
1	Performance	High	System responds within 3 seconds
2	Usability	High	Supports natural language commands
3	Reliability	High	Operates without failure for 1 hour
4	Security	Medium	Access restricted to authorized users
5	Scalability	Medium	Expandable to more commands
6	Adaptability	Medium	Compatible with different camera models

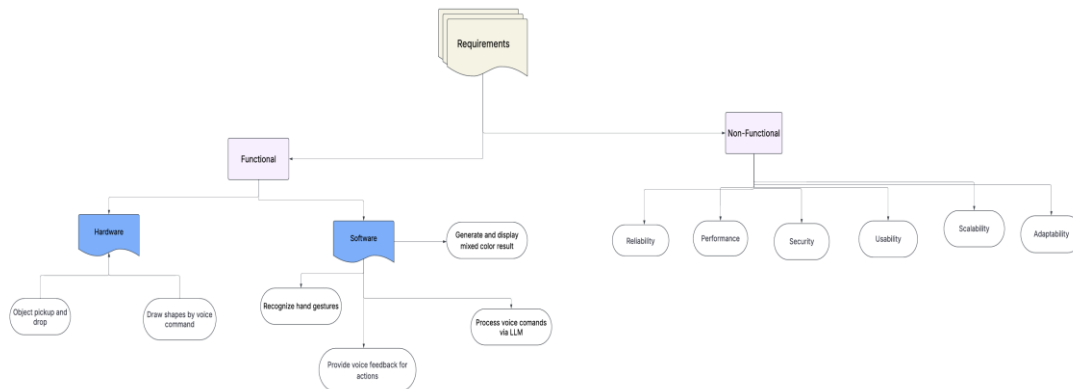


Image 2: Simplified chart of functional and non-functional requirements

4 Testing

4.1 Testing Schedule

The testing schedule ensures reliability and performance:

4.1.1 Unit Testing

- Each software component, namely UI, AI model, and ROS integration, shall be individually tested.
- Ensure that all functions work properly before integration.
- Testing shall be done with automated test frameworks, for example, pytest for Python-based components.

4.1.2 Integration Testing

- Tests the interaction between software modules and hardware.
- AI predictions control the robotic arm.
- ROS Gazebo simulations are performed before actual deployment.

4.1.3 System Testing

- Confirms whether the system meets all kinds of functional and non-functional requirements.
- Includes performance, usability, and stress testing.
- Performed with real-world input to test for complete teleoperation capability.

4.1.4 Acceptance Testing

- Ensure that the project meets requirements set by the stakeholder.
- Carried out with real users to confirm its usability and effectiveness.

4.2 Test Cases

The following are sample test cases for the major components:

4.2.5 UI Testing

Test Case ID	Description	Input	Expected Output
UI-01	Button responsiveness	Click button	Immediate visual resp
UI-02	Command execution	Enter teleoperation command	Robot arm moves corr

4.2.6 AI Model Testing

Test Case ID	Description	Input	Expected Output
AI-01	Object classification	Image input	Correct object label
AI-02	Path planning	Obstacle positions	Optimal movement path

4.2.7 ROS Integration Testing

Test Case ID	Description	Input	Expected Output
ROS-01	Command execution	Move command	Robot moves in ROS simulation
ROS-02	Sensor data processing	Sensor input	Correct data output

4.3 Testing Schedule Timeline

Testing Phase	Start Date	End Date	Duration
Unit Testing	25-Feb-2025	01-March-2025	7 days
System Testing	01-March-2025	05-March-2025	4 days
Integration Testing	05-March-2025	10-March-2025	5 days

Testing is aligned with the project plan and divided into sprints:

The testing schedule ensures the reliability and performance of the intelligent teleoperation system through structured phases, including unit, integration, system, and acceptance testing. Unit testing verifies individual components such as the user interface, AI model, and robotic system integration using automated frameworks. Integration testing examines interactions between software modules and hardware, ensuring smooth communication and functionality. System testing evaluates overall performance, usability, and stress handling under real-world conditions, while acceptance testing confirms that the system meets user expectations. Enhancements include expanding AI model testing for gesture and voice recognition, introducing latency and security assessments, and conducting endurance tests to ensure sustained operation. These improvements will optimize system responsiveness, security, and overall efficiency.

Task Number	Task Title	Task Owner	Start Date	End Date	Duration (Days)	Progress (%)
1.1	UI Development	Team A	2025-02-05 00:00:00	2025-02-10 00:00:00	5	0
1.2	UI Functional Testing	Team A	2025-02-10 00:00:00	2025-02-15 00:00:00	5	0
1.3	UI Performance Testing	Team A	2025-02-15 00:00:00	2025-02-20 00:00:00	5	0
2.1	AI Model Development	Team B	2025-02-20 00:00:00	2025-02-25 00:00:00	5	0
2.2	AI Model Testing	Team B	2025-02-25 00:00:00	2025-03-01 00:00:00	5	0
3.1	ROS Integration	Team C	2025-03-01 00:00:00	2025-03-05 00:00:00	5	0
3.2	ROS Functional Testing	Team C	2025-03-05 00:00:00	2025-03-10 00:00:00	5	0
4.1	Hardware Setup	Team D	2025-03-10 00:00:00	2025-03-15 00:00:00	5	0
4.2	Hardware Calibration	Team D	2025-03-15 00:00:00	2025-03-20 00:00:00	5	0
5.1	Real-time Teleoperation Testing	Team E	2025-03-20 00:00:00	2025-03-25 00:00:00	5	0
6.1	Unit Testing	QA Team	2025-03-25 00:00:00	2025-04-01 00:00:00	5	0
6.2	System Testing	QA Team	2025-04-01 00:00:00	2025-04-05 00:00:00	4	0
6.3	Integration Testing	QA Team	2025-04-05 00:00:00	2025-04-10 00:00:00	5	0

4.4 Tools and Resources

- **Testing Tools:** PyTest, ROS Gazebo, Manual UI Testing.
- **Platforms:** ROS, Python, OpenCV for AI Testing.
- **Hardware:** Dobot Robotic Arm and Sensors.

5 Project Planning

This section of the proposal highlights and describes which management structure and methodologies are best in terms of improving team collaboration, communication, and efficiency. Additionally, tools used for enhancing project management and planning will also be discussed in detail, with graphical representation of milestones to be achieved according to timestamps represented in the form of a Gantt chart.

5.1 Project Management Structure/Methodology

Management structures and methodologies help in achieving objectives in a timely and effective fashion. Deciding which management structure methodology to use depends entirely on the project's scope, complexity, specification, and requirements. Therefore, the next step in our project planning was to decide on which approach will be the most appropriate and potent.

Listed below are some of the strategies that might be suitable for the completion of our project:

- 1. Agile Methodology**
- 2. Waterfall Methodology**
- 3. Hybrid Methodology**
- 4. Lean Methodology**

5.1.1 Agile Methodology

Agile project management focuses on being flexible and cooperative. It emphasizes adaptability, ensuring process improvement and timely completion. Agile allows teams to self-organize according to the project's needs, leaving room for progress reviews, quick iterations, and continuous improvement. This methodology is ideal for projects with unfixed requirements, where feedback is crucial, mostly in software development, electronics, and R&D.

5.1.2 Waterfall Methodology

The Waterfall methodology is the most conventional and suggests a linear and sequential approach to handling projects, where the primary phase or task must be completed before starting the next phase. Unlike Agile, it is slower and less adaptable. This model is only suitable when the end results are fixed, and requirements or specifications do not change. Otherwise, deviation could compromise the entire project.

5.1.3 Hybrid Methodology

As the name suggests, Hybrid methodology combines Agile and Waterfall approaches. By blending the strengths of both

methodologies, their disadvantages are effectively eliminated. This method provides a balance of structure and flexibility. It is best suited for projects that require both predictability (fixed goals) and adaptability (iterative improvement). Hybrid methodology is especially ideal for projects involving both hardware and software components.

5.1.4 Lean Methodology

Lean methodology emphasizes achieving the best end results by eliminating waste and optimizing processes through efficient resource utilization. By identifying and prioritizing true values of the project, unnecessary steps are minimized. The workflow is then streamlined to ensure smooth operations. This approach is most suitable for projects with limited resources, focusing only on the output rather than the process.

5.2 Project Planning Tools

Project planning tools provide a platform for all entities involved in project management to collaborate and maintain a common workspace. Supervisors can provide timely feedback, monitor progress, and help teams rectify mistakes and debug issues. By periodically updating or uploading progress, teams can work on future improvements and roll back to any previous state if needed.

5.2.1 Jira

Jira, developed by Atlassian, is one of the most commonly used project management tools. It features task and issue tracking, aiding in managing projects and tracking bugs and feature requests. Jira provides custom workflows, enabling configuration to match project needs. Using its collaboration tools, supervisors can assign tasks and provide feedback. Jira supports numerous project management methodologies, with Agile being the most common. Additionally, it allows automation and integration with tools like GitHub, Slack, and CI/CD pipelines.

5.2.2 GitLab

GitLab is a versatile tool for recording updates, testing new features,

and revision control. It uses Git (version control system) to provide multi-user access, making project management efficient and automating deployment. Unlike GitHub, GitLab offers an end-to-end DevOps solution. It supports self-hosting on private servers and provides built-in security services and access control.

Task Title	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	Week 10	Week 11
UI Development	1	1	0	0	0	0	0	0	0	0	0
UI Functional Testing	0	0	1	1	0	0	0	0	0	0	0
UI Performance Testing	0	0	0	0	1	0	0	0	0	0	0
AI Model Development	0	2	2	0	0	0	0	0	0	0	0
AI Model Testing	0	0	0	2	2	0	0	0	0	0	0
ROS Integration	0	0	3	3	0	0	0	0	0	0	0
ROS Functional Testing	0	0	0	0	3	0	0	0	0	0	0
Hardware Setup	0	0	0	0	0	4	4	0	0	0	0
Hardware Calibration	0	0	0	0	0	0	4	4	0	0	0
Real-time Teleoperation Testing	0	0	0	0	0	0	0	5	5	0	0
Unit Testing	0	0	0	0	0	0	0	0	6	6	0
System Testing	0	0	0	0	0	0	0	0	0	6	6
Integration Testing	0	0	0	0	0	0	0	0	0	0	6

6 References

<https://www.sciencedirect.com/science/article/pii/S0952197614001286>
<https://ieeexplore.ieee.org/abstract/document/8643076/>
<http://www.ieomsociety.org/toronto2019/papers/436.pdf>
<https://www.sciencedirect.com/science/article/pii/S0167639304000706>
<https://ieeexplore.ieee.org/abstract/document/479281/>
<https://ieeexplore.ieee.org/abstract/document/10035484/>
<https://ieeexplore.ieee.org/abstract/document/10035484/>
<https://ieeexplore.ieee.org/abstract/document/4343985/>
https://books.google.com/books?hl=en&lr=&id=ZEfqCAAQBAJ&oi=fnd&pg=PA9&dq=teleoperation+robot&ots=1YPRSU6hZM&sig=1bkgcJTcHHVYnxSXE_4W2r-nGrM